Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность системы управления информацией

Выполнил:
Студент группы АС-68/24
Сендер К.О.
Проверил:
А. А. Крощенко
доцент кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: на практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 7

- **Telco Customer Churn**
- Предсказать, откажется ли клиент от услуг телеком-оператора
- **Задания:**
  - загрузите данные, обработайте категориальные признаки и пропуски;
  - разделите данные на обучающую и тестовую выборки;
  - обучите модели **k-NN**, **Decision Tree** и **SVM**;
  - сравните модели по метрике **F1-score** для класса "отток";
  - сделайте вывод о применимости каждого из методов для решения бизнес-задачи по удержанию клиентов.

Код программы:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay)

# 1. Загрузка данных
df = pd.read_csv("Telco-Customer-Churn.csv")
print("Размер данных:", df.shape)
print("Ушли клиентов:", (df["Churn"] == "Yes").sum(), "из", len(df))

# 2. Обработка пропусков и категориальных признаков
df = df.drop(columns=["customerID"])
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
print("Пропуски в TotalCharges:", df["TotalCharges"].isnull().sum())
df["TotalCharges"] = df["TotalCharges"].fillna(0)
df["Churn"] = (df["Churn"] == "Yes").astype(int)

X = pd.get_dummies(df.drop(columns=["Churn"]), drop_first=True, dtype=int)
```

```
y = df["Churn"]

# 3. Разделение на выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=2024, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Обучающая выборка:", X_train.shape[0], "| Тестовая выборка:",
X_test.shape[0])

# 4. Исследование k-NN
k_values = range(1, 102, 2)  # до 101, чтобы не было на границе
f1_by_k = []
acc_by_k = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    pred = knn.predict(X_test)
    f1_by_k.append(f1_score(y_test, pred))
    acc_by_k.append(accuracy_score(y_test, pred))

best_k = k_values[f1_by_k.index(max(f1_by_k))]

print("\nПодбор k для k-NN")
for k in [1, 5, 11, 21, 31, 41, 51, 61, 71, 81, 91, 101, best_k]:
    if k in k_values:
        idx = k_values.index(k)
        print(f"k = {k}: F1-score = {f1_by_k[idx]:.3f}")

print(f"Лучшее k: {best_k}")

plt.figure(figsize=(10, 6))
plt.plot(k_values, f1_by_k, marker="o", label="F1-score (отток)")
plt.plot(k_values, acc_by_k, marker="s", label="Accuracy")
plt.axvline(best_k, color="red", linestyle="--", label=f"Лучшее k = {best_k}")
plt.xlabel("Количество соседей k")
plt.ylabel("Значение метрики")
plt.title("Качество k-NN при разном количестве соседей")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("01_knn_k.png")
plt.close()

# 5. Обучение трех моделей
models = {
    "k-NN": KNeighborsClassifier(n_neighbors=best_k),
    "Decision Tree": DecisionTreeClassifier(max_depth=6, random_state=2024),
    "SVM": SVC(kernel="linear", random_state=2024),
}

results = []
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))

for ax, (name, model) in zip(axes, models.items()):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    results.append({
        "Модель": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
```

```
            "F1-score": f1_score(y_test, pred),
        })

        cm = confusion_matrix(y_test, pred)
        ConfusionMatrixDisplay(cm, display_labels=["Остался", "Отток"]).plot(
            ax=ax, cmap="Blues", values_format="d", colorbar=False
        )
        ax.set_title(name)
        ax.set_xlabel("Предсказано")
        ax.set_ylabel("Реально")

plt.tight_layout()
plt.savefig("02_confusion_matrices.png")
plt.close()

results = pd.DataFrame(results).set_index("Модель").round(3)

# 6. Сравнение моделей по F1-score
plt.figure(figsize=(8, 5))
bars = plt.bar(results.index, results["F1-score"], color=["#4C72B0", "#55A868",
"#C44E52"])
for bar, value in zip(bars, results["F1-score"]):
    plt.text(bar.get_x() + bar.get_width() / 2, value + 0.01, f"{value:.3f}",
ha="center")
plt.ylim(0, 1)
plt.ylabel("F1-score (класс «отток»)")
plt.title("Сравнение моделей по F1-score")
plt.tight_layout()
plt.savefig("03_f1_comparison.png")
plt.close()

print("\nСравнение моделей (F1-score для класса 'отток')")
for name, row in results.sort_values("F1-score", ascending=False).iterrows():
    print(f"{name}: F1-score = {row['F1-score']:.3f}, Accuracy =
{row['Accuracy']:.3f}")

best_model = results["F1-score"].idxmax()
print(f"Лучшая модель: {best_model}")
print(f"Её F1-score: {results.loc[best_model, 'F1-score']:.3f}")
```

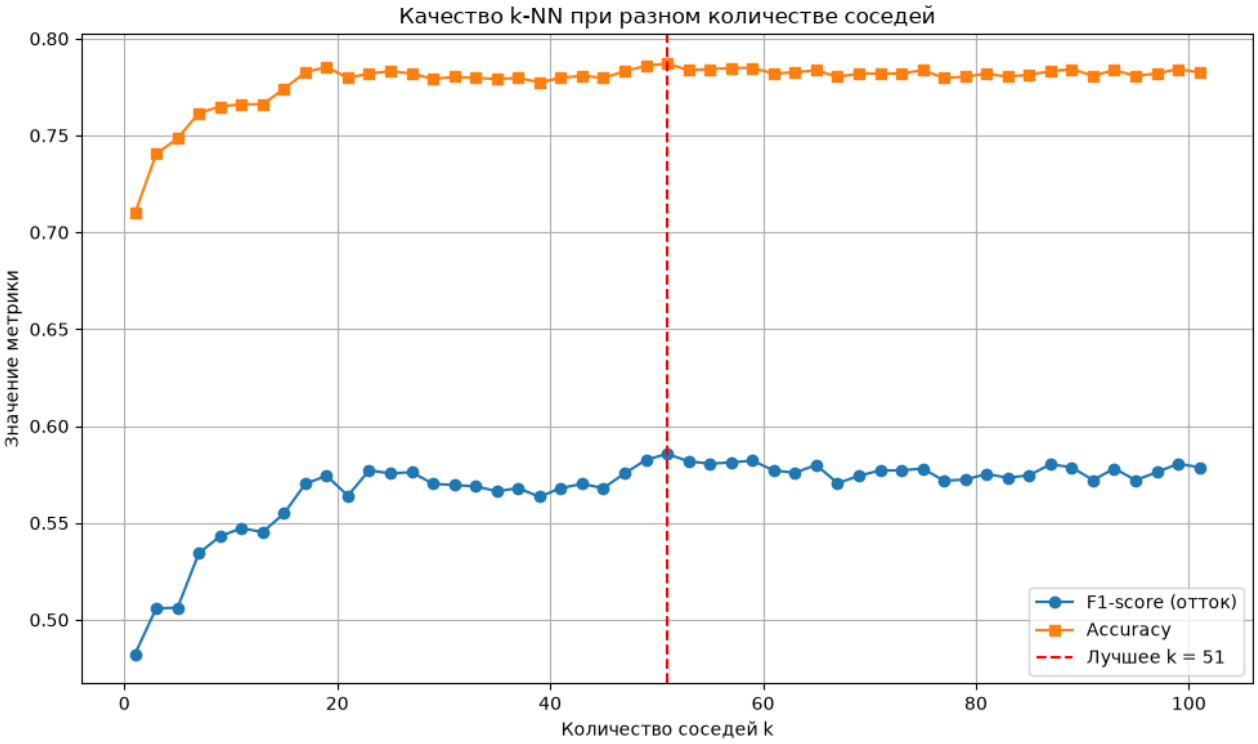
Результаты выполнения программы:

```
Размер данных: (7043, 21)
Ушли клиентов: 1869 из 7043
Пропуски в TotalCharges: 11
Обучающая выборка: 5282 | Тестовая выборка: 1761

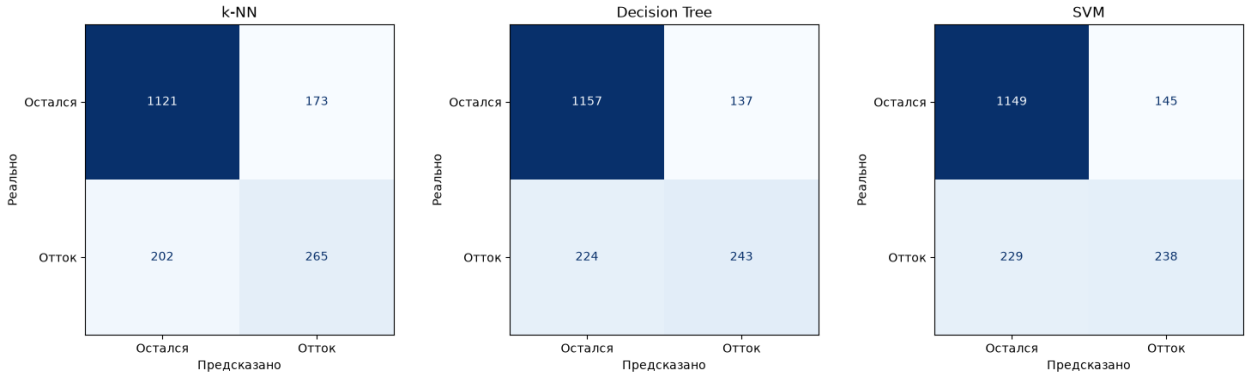
Подбор k для k-NN
k = 1: F1-score = 0.482
k = 5: F1-score = 0.506
k = 11: F1-score = 0.547
k = 21: F1-score = 0.564
k = 31: F1-score = 0.570
k = 41: F1-score = 0.568
k = 51: F1-score = 0.586
k = 61: F1-score = 0.577
k = 71: F1-score = 0.577
k = 81: F1-score = 0.575
k = 91: F1-score = 0.572
k = 101: F1-score = 0.579
k = 51: F1-score = 0.586
Лучшее k: 51

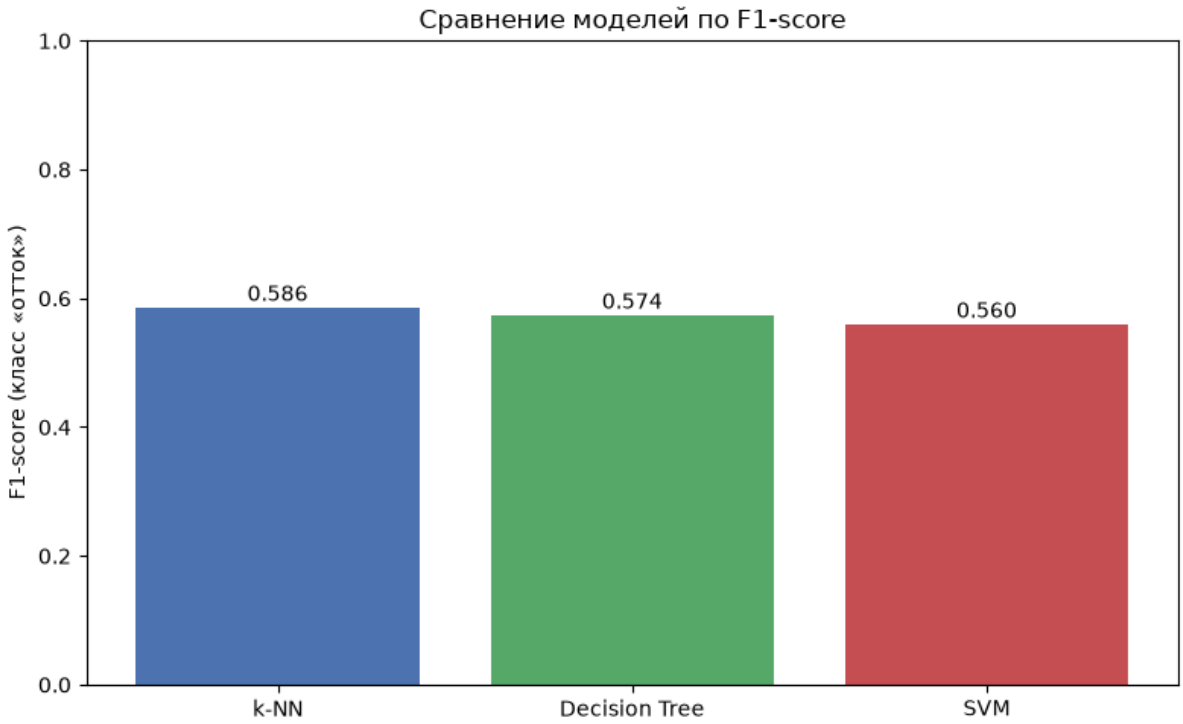
Сравнение моделей (F1-score для класса 'отток')
k-NN: F1-score = 0.586, Accuracy = 0.787
Decision Tree: F1-score = 0.574, Accuracy = 0.795
SVM: F1-score = 0.560, Accuracy = 0.788
Лучшая модель: k-NN
Её F1-score: 0.586
```

Исследование k-NN при разном количестве соседей:

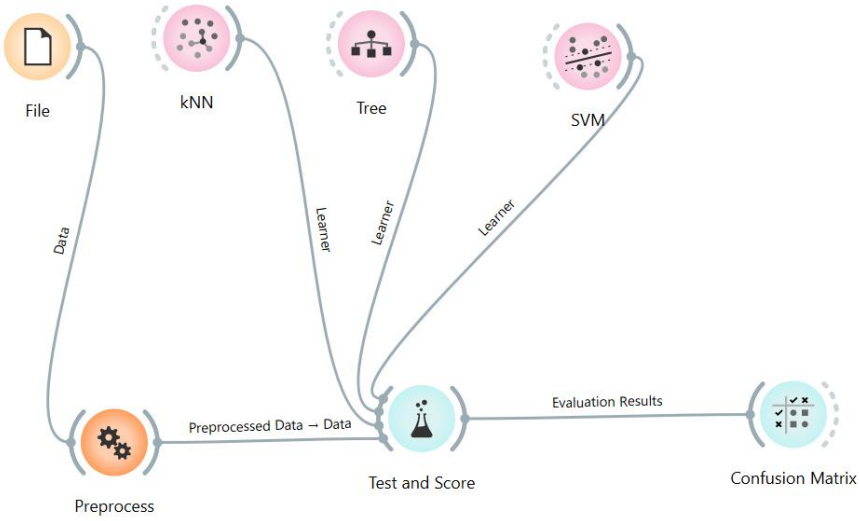


Лучшее значение F1-score = 0.586 получено при k = 51.





Orange:



Test and Score - Orange

Cross validation
Number of folds: 5
Stratified
Cross validation by feature
Random sampling
Repeat train/test: 10
Training set size: 66 %
Stratified
Leave one out
Test on train data
Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
kNN	0.780	0.776	0.748	0.757	0.776	0.345
Tree	0.819	0.787	0.782	0.779	0.787	0.432
SVM	0.297	0.666	0.682	0.717	0.666	0.264

Compare models by: Area under ROC curve Negligible diff.: 0.1

	kNN	Tree	SVM
kNN		0.001	0.997
Tree	0.999		0.998
SVM	0.003	0.002	

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.

7043 | - | 7043 | 3×7043

Confusion Matrix - Orange

Learners
kNN
Tree
SVM

Clicking on cells or in headers outputs the corresponding data instances
Ok, got it

how: Number of instances

Predicted

Actual	No	Yes	Σ
No	4547	627	5174
Yes	873	996	1869
Σ	5420	1623	7043

Output
Predictions
Probabilities
Apply Automatically

Select Correct | Select Misclassified | Clear Selection

3×7043 | - | 7043

Вывод:

На практике сравнил работу алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научился подбирать гиперпараметры моделей и оценивать их влияние на результат.

Для задачи удержания клиентов лучше всего подходит k-NN (k = 51): у неё наибольший F1-score (0.586), и она находит больше всего уходящих клиентов. Дерево решений показало немного меньший F1-score (0.574), но наилучшую общую точность Accuracy (0.795). Метод опорных векторов (SVM) с линейным ядром оказался самым быстрым в обучении, однако его F1-score оказался наименьшим (0.560). Сделан вывод, что для данной бизнес-задачи (где важно не пропустить уходящего клиента) лучше всего использовать k-NN.